

Ahmed Abdelrahman

Cairo, Egypt (UTC+3) | +201024180920 | ahmedabdelrhaman232@gmail.com |
<https://www.linkedin.com/in/ahmed-abdelrahman-89bbb3203/> | <https://kennycandra.dev>

Technical Skills

Languages

TypeScript, Rust, Python

Backend

Node.js, Express.js, REST APIs, Event-Driven Architecture, Async Processing

Databases

PostgreSQL, MongoDB, Redis

Async & Messaging

BullMQ, Dead-Letter Queues, Idempotent Processing

Infrastructure

Docker, GitHub Actions CI/CD, AWS ECS Fargate, DigitalOcean

Work Experience

Vogel Edits

Full-Stack Engineer (Adobe Extensions)

- Architected a **Node.js** sidecar process to handle **AI inference** outside Adobe's sandboxed CEP runtime, enabling scalable **microservices** communication via local **HTTP API** for real-time results.
- Designed and built **REST APIs** and background services for **ONNX** model inference, implementing **event-driven** patterns to decouple frontend UI from backend processing.
- Implemented **caching** strategies and **async** processing pipelines to optimize performance of **AI-powered** tools across production Adobe extensions.

Freelance

Full-Stack Engineer

- Designed and built **REST APIs** using **Node.js** and **Express.js** with **TypeORM** for type-safe **PostgreSQL** access, delivering scalable backend services for a tourism booking platform.
- Optimized **database** performance using **PostgreSQL** indexing strategies and **EXPLAIN ANALYZE** to achieve sub-100ms **API** response times under concurrent load.
- Implemented secure payment **API** integration with idempotent transaction handling and webhook verification, reducing checkout friction by 40%.

RootCluster

Backend Engineer

- Designed and deployed a distributed, **event-driven** log ingestion pipeline on **AWS ECS Fargate** using **BullMQ** for **job queuing**, **Redis** for **caching** and rate limiting, and **PostgreSQL** (pgvector) for similarity search — processing 1M+ events/hour with sub-second latency.
- Implemented robust **async** processing patterns: producer-consumer queues for log normalization, **worker pools** with configurable concurrency, and **priority queues** for urgent events vs. batch backfill jobs.
- Built **dead-letter queues (DLQs)** with retry policies (exponential backoff, max attempts) to handle malformed or failing log entries without pipeline blockage.
- Designed **idempotent** event processing using **Redis**-backed dedup keys and at-least-once delivery semantics through BullQueue's job completion callbacks.
- Reduced **ONNX** embedding time from 20+ minutes to under 1 minute via batch processing, **Redis** caching, and **worker autoscaling**.
- Set up full **CI/CD** with **GitHub Actions** OIDC, **ECR**, **SSM Parameter Store**, **RDS PostgreSQL**, and **ElastiCache**.

Medical CRM

Full-Stack Engineer

- Designed and implemented **REST APIs** using **Node.js** and **Express.js** for a full-stack CRM handling diagnoses, prescriptions, and patient records.
- Implemented **JWT** auth with access/refresh token rotation and **RBAC** across three permission levels, securing backend **API** endpoints.
- Optimized **database** queries for 10K+ monthly records using composite indexes and **PostgreSQL** window functions.

Projects

RootCluster — AI-Powered Log Analysis Platform [rootcluster.dev](#)

Tech Stack: Node.js, TypeScript, PostgreSQL (pgvector), Redis, BullMQ, ONNX, AWS ECS Fargate, Docker, GitHub Actions

- Designed and deployed a distributed, **event-driven** log ingestion pipeline on **AWS ECS Fargate** using **BullMQ** for **job queuing**, **Redis** for **caching** and rate limiting, and **PostgreSQL** (pgvector) for similarity search — processing 1M+ events/hour with sub-second latency.
- Implemented robust **async** processing patterns: producer-consumer queues for log normalization, **worker pools** with configurable concurrency, and **priority queues** for urgent events vs. batch backfill jobs.
- Built **dead-letter queues (DLQs)** with retry policies (exponential backoff, max attempts) to handle malformed or failing log entries without pipeline blockage.
- Designed **idempotent** event processing using **Redis**-backed dedup keys and at-least-once delivery semantics through BullQueue's job completion callbacks.
- Reduced **ONNX** embedding time from 20+ minutes to under 1 minute via batch processing, **Redis** caching, and **worker autoscaling**.
- Implemented centroid-based **pgvector** clustering to unify cross-service failures into single root causes.
- Set up full **CI/CD** with **GitHub Actions** OIDC, **ECR**, **SSM Parameter Store**, **RDS PostgreSQL**, and **ElastiCache**.

Medical CRM — AI-Enhanced Prescription Platform

Tech Stack: React, Node.js, PostgreSQL

- Built full-stack CRM handling diagnoses, prescriptions, and patient records.
- Implemented **JWT** auth with access/refresh token rotation and **RBAC** across three permission levels.
- Designed drug interaction validation across 500+ medications using graph-based **PostgreSQL** lookup.
- Optimized dashboard queries for 10K+ monthly records using composite indexes and window functions.

Coursework

- Meta Certified Frontend Developer — Meta (2024)
- Complete SQL Databases Bootcamp: Zero to Mastery — Udemy
- Master the Coding Interview: Data Structures & Algorithms — Udemy
- Node.js: The Complete Guide — Udemy
- Grokking Algorithms — Aditya Bhargava
- Designing Data-Intensive Applications — Martin Kleppmann